Labo 16 — Questions

Question 1 [Compréhension]

Expliquez la différence entre `requests` et `limits` pour la mémoire et pour le CPU : qui lit chacune (scheduler, kubelet, cgroup), et que se passe-t-il quand un conteneur dépasse la **limit** mémoire d'une part, la limit CPU d'autre part ?

Question 2 [Prédiction]

Trois Pods sur un nœud saturé en mémoire : `db` avec `requests = limits = 512Mi`, `api` avec `requests 64Mi / limits 256Mi`, `batch` sans `resources`. Donnez la classe de QoS de chacun, l'ordre dans lequel le kubelet les évince, et expliquez pourquoi ce n'est pas l'ordre que l'équipe souhaiterait pour `batch` s'il s'agissait d'un traitement comptable critique.

Question 3 [Diagnostic]

```
$ kubectl get pod glouton
NAME      READY   STATUS      RESTARTS      AGE
glouton   0/1     OOMKilled   1 (14s ago)   16s
```

Donnez les deux commandes qui confirment la cause et la valeur dépassée, expliquez pourquoi `RESTARTS` augmente, et dites ce qui, dans ce cas, est à corriger : la limit, l'application, ou les deux.

Question 4 [Diagnostic]

Après `kubectl scale deployment api --replicas=6`, trois Pods restent `Pending`. `kubectl describe node | grep -A6 'Allocated resources'` montre `memory 3800Mi (95%)` en requests. Expliquez, proposez trois corrections différentes, et dites laquelle est la plus honnête.

Question 5 [Analyse]

Un LimitRange avec `defaultRequest: {memory: 32Mi}`, `default: {memory: 128Mi}` et `max: {memory: 1Gi}` est posé dans le namespace. Décrivez ce qui arrive à : (a) un Pod sans `resources` ; (b) un Pod avec `limits.memory: 4Gi` ; (c) un Pod avec `requests.memory: 64Mi` et sans limit. Pourquoi l'équipe plateforme impose-t-elle ces objets ?

Question 6 [Prédiction]

Une ResourceQuota fixe `requests.memory: 1Gi` et `pods: 10` sur le namespace. Un Deployment à trois réplicas avec `requests.memory: 64Mi` tourne. Vous appliquez un second Deployment à huit réplicas de `128Mi` chacun. Que montre `kubectl get pods` pour ce second Deployment, que dit `kubectl describe deployment`, et où lit-on le refus ?

Question 7 [Compréhension]

Pourquoi `kubectl logs api-xxx` affiche-t-il parfois des logs sains pour un Pod qui vient de redémarrer, alors que la cause du redémarrage est dans `kubectl logs api-xxx --previous` ? Que montre `kubectl get pod -o jsonpath='{.status.containerStatuses[0].lastState}'` ?

Question 8 [Analyse]

L'image de l'API est distroless (labo 05) : pas de shell, pas de `wget`. Un Pod répond lentement et vous voulez voir ses processus et tester son port depuis l'intérieur. Donnez la commande `kubectl`

`debug` complète, expliquez ce que `--target` apporte, et dites pourquoi cette approche est préférable à « ajouter un shell dans l'image au cas où ».

Question 9 [Diagnostic]

```
$ kubectl get secrets --as=system:serviceaccount:lab16:reader
Error from server (Forbidden): secrets is forbidden: User "system:serviceaccount:lab16:reader"
cannot list resource "secrets" in API group "" in the namespace "lab16"
```

Le ServiceAccount `reader` a un Role sur `pods` et `pods/log`. Expliquez chaque partie du message, dites si c'est un bug ou le comportement voulu, et donnez la règle à ajouter si `reader` devait lire les ConfigMaps (pas les Secrets).

Question 10 [Analyse]

Comparez Role + RoleBinding avec ClusterRole + ClusterRoleBinding, puis le cas mixte ClusterRole + RoleBinding. Pour chacun, donnez un usage concret dans une entreprise, et expliquez pourquoi les ClusterRoles `view` / `edit` / `admin` sont presque toujours utilisés via des RoleBindings.

Question 11 [Prédiction]

```
kubectl apply -f netpol-api.yaml       # podSelector app=api, ingress from podSelector app=web,
port 8080
kubectl exec client -- wget -qO- --timeout=3 http://api:8080/actuator/health
kubectl label pod client app=web
kubectl exec client -- wget -qO- --timeout=3 http://api:8080/actuator/health
```

Prédisez les deux résultats, expliquez ce que « sélectionné par une politique » change, et dites ce qui se passerait sur un cluster dont le CNI n'applique pas les NetworkPolicies.

Question 12 [Analyse]

L'équipe sécurité demande que `db` ne soit joignable que par `api`, que `api` ne soit joignable que par `web` et par Traefik, et que rien d'autre ne change. Écrivez le principe des politiques nécessaires (sans YAML complet), dites où se cache la difficulté pour Traefik (indice : namespace), et pourquoi on commence souvent par une politique « tout refuser ».

Question 13 [Analyse]

Un Pod de l'API a été lancé avec le ServiceAccount `default` et, suite à une faille, un attaquant obtient l'exécution de code dedans. Qu'obtient-il grâce au jeton monté dans le Pod sur un cluster où `default` n'a aucun RoleBinding ? Et sur un cluster où un administrateur pressé avait lié `default` à `cluster-admin` « pour que le déploiement marche » ? Quelles deux mesures du labo auraient réduit le risque ?